IE437 · DATA-DRIVEN DECISION MAKING AND CONTROL — LECTURE 8

Value-Based Reinforcement Learning

Learning to act when the model is gone

Jinkyoo Park · KAIST

From the Bellman equation you can *solve* to the one you can only *sample*

HANDOFF

The handoff — what Lecture 7 left us

Lecture 7 owned the world. This lecture takes it away and keeps the equation.

Where we are — the OR lineage, made data-driven

STAGES **dynamic** ○ —— ● | MODEL · CROSSING model-based → **data-driven** ○ —— ● | AGENTS **single agent** ● —— ○

	LINEAGE A · OR / DYNAMIC PROGRAMMING	LINEAGE B · CONTROL THEORY
MODEL-BASED (ORIGIN)	**MDP & Dynamic Programming** LECTURE 7	**Optimal Control / Planning** LECTURE 9
DATA-DRIVEN (EXTENSION)	**Value-Based RL** LECTURE 8	**Policy-Based RL** LECTURE 10

We stay in the same cell of the big map — **single agent, multiple stages** — and move along **one axis only**: from a world we are *handed* (P, R known) to a world we can only *sample*. Lecture 10 will perform the identical move on the control lineage.

What Lecture 7 gave us, stated as one object — the **Bellman optimality equation**:

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\Big]$$

And two ways to solve it — value iteration, policy iteration. Both are exact. Both are **useless without the two highlighted terms**.

What we keep, and what we lose

The Bellman equation survives. **The model does not.**

Everything else follows from removing P and R. Three things break the moment they vanish:

- **The expectation.** $\sum_{s'} P(s' \mid s, a)[\cdots]$ cannot be computed — we never see P, only one sampled s' at a time.
- **The improvement step.** Acting greedily on V needs P to look one step ahead. Without it, V alone cannot tell us *what to do*.
- **The table.** Even with samples, a Q-value per state–action pair does not fit in memory — nor generalize — once states are images.

Each break is one question of today. We never replace the Bellman equation; we replace the *model inside it* — first by samples, then by a function.

The translation table — our coordinate system for today

Every object of Lecture 7 has a *sampled* counterpart. Keep this in sight.

LECTURE 7 (MODEL-BASED)	LECTURE 8 (SAMPLED)	WHAT REPLACES THE MODEL
expectation $\sum_{s'} P(s' \mid s,a)[\cdots]$	one transition (s,a,r,s')	a sample average / a bootstrap
policy evaluation (solve with P, R)	MC or TD prediction	returns, or a one-step estimate
greedy on V: $\arg\max_a \sum_{s'} P[\cdots]$	greedy on Q: $\arg\max_a Q(s,a)$	store Q, not V
value iteration	Q-learning	the $\max$ inside a sampled backup
full backup (all successors)	sample backup (one successor)	the law of large numbers
tabular, exact	function approximation (DQN)	a network $Q(s,a;w)$

Read column 3. Nothing here is a new *principle* — it is the same Bellman backup, with the one piece we don't own quietly swapped out. The art is in **what we swap it for** without the whole thing collapsing.

One honest detour — why not just learn the model?

The most literal idea: estimate P, R from data, then run Lecture 7.

$$\hat{P}(s' \mid s, a) = \frac{\#(s, a, s')}{\#(s, a)}, \qquad \hat{R}(s, a, s') = \text{average } r \text{ over } (s, a, \cdot, s')$$

Plug in, solve the MDP. This is **model-based RL**, and it is perfectly valid.

But notice the waste: to choose actions we only ever need Q^*, yet here we first estimate a *whole transition kernel* — vastly more parameters than the thing we actually use. Why estimate P and R at all,

when we could estimate Q^* directly?

That question — *skip the model, estimate the value* — is the entire model-free programme. The rest of today walks it. (The model returns, deliberately, in Lecture 11.)

The roadmap — four questions

One question per Act. This strip returns at every transition — watch the highlight move.

Q1 Evaluate? → Q2 Improve? → Q3 Explore? → Q4 Scale?

- **Q1 — How do we evaluate a policy with no model?** Sample the whole return, or **bootstrap** from one step. *(MC vs. TD; Sutton, 1988)*
- **Q2 — How do we improve with no model?** Greedy-on-V needs P; so learn Q, and pay for exploration with ε-greedy.
- **Q3 — Whose value are we learning?** On-policy (**SARSA**) vs. off-policy (**Q-learning**) — the cliff decides. *(Watkins, 1989)*
- **Q4 — Can it scale past a table?** Function approximation, the deadly triad, and the two tricks of **DQN**. *(Mnih et al., 2013/2015)*

ACT 1

Evaluation without a model

Q1. Fix a policy. With no P and no R, how good is it?

The arena — from MDP to experience

Q1 Evaluate? → Q2 Improve? → Q3 Explore? → Q4 Scale?

We no longer query a model. We *act*, and a stream comes back:

$$s_1, a_1, r_2, s_2, a_2, r_3, s_3, \ldots, s_T \qquad \text{(one episode)}$$

- the agent chooses $a_t = \pi(s_t)$ — *decision*,
- the environment returns r_{t+1}, s_{t+1} — *the only window onto* P, R,
- from this stream we must recover Q^* and act on it — *learning*.

Start with the humblest sub-task: **prediction**. Fix a policy π; estimate its value V^π. No control yet — just: how good is this behavior? Everything harder is built on getting this right.

The quantity, and the two ways to estimate it

The definition has not changed since Lecture 7 — only our access to it:

$$V^\pi(s) = \mathbb{E}_\pi\big[G_t \mid s_t = s\big] = \underbrace{\mathbb{E}_\pi\Big[\textstyle\sum_{k\geq 0}\gamma^k r_{t+k+1} \,\Big|\, s_t = s\Big]}_{\text{the full return}} = \underbrace{\mathbb{E}_\pi\big[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s\big]}_{\text{one step, then bootstrap}}$$

The same value, written two ways. Each writing is a recipe for estimating it from samples:

- **Monte Carlo** reads the *left* form: wait for the episode to end, average the realized return G_t.
- **Temporal Difference** reads the *right* form: take one step, then **trust your own current estimate** of the rest.

Sample the whole thing, or sample one step and bootstrap — **the deepest fork in all of value-based RL.**

Monte Carlo prediction — average what actually happened

Visit s_t, watch the episode finish, collect the realized return $G_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{T-t-1} r_T$. Then nudge:

$$V(s_t) \leftarrow V(s_t) + \alpha\big[\underbrace{G_t}_{\text{target}} - V(s_t)\big]$$

- **Unbiased.** $\mathbb{E}[G_t] = V^\pi(s_t)$ exactly — the target is the real thing, not a guess.
- **No model, no bootstrap.** Each state's estimate stands alone; we never lean on neighbors.
- **The price:** must wait for the terminal state (no online updates), and the return of a long episode is a high-variance single draw — one unlucky tail corrupts the estimate.

MC also *wastes* the structure of the MDP: it learns each state separately and never reuses the fact that s leads to s'. That wasted structure is exactly what TD reclaims.

Temporal Difference — learn a guess from a guess

Don't wait. After a *single* transition (s_t, r_{t+1}, s_{t+1}), update:

$$V(s_t) \leftarrow V(s_t) + \alpha\big[\underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{TD target}} - V(s_t)\big]$$

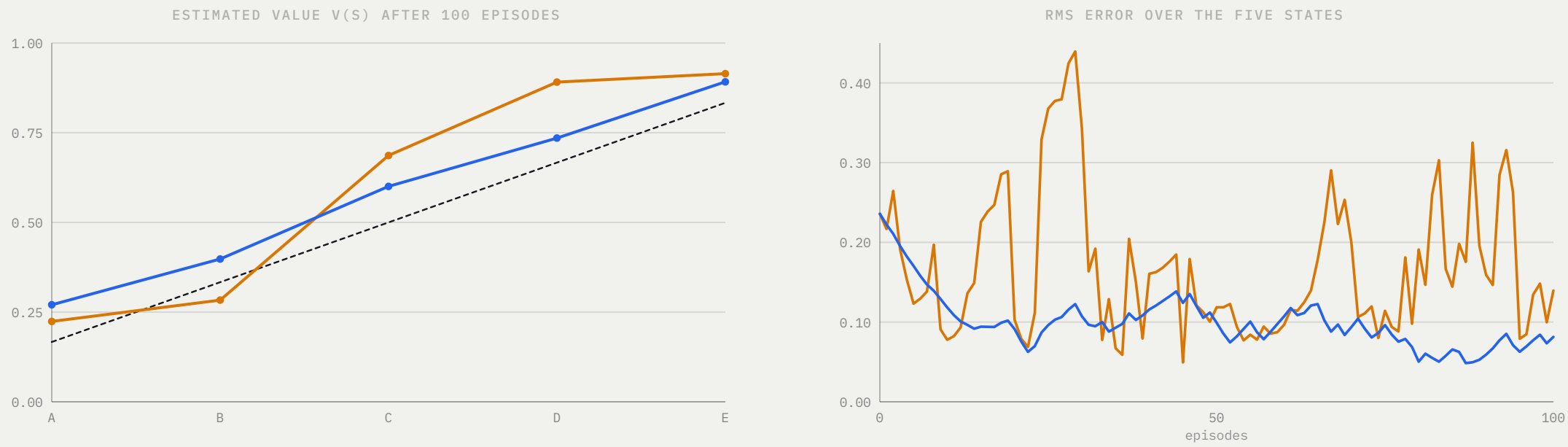
The target contains $V(s_{t+1})$ — **our own current estimate**. We update a guess toward a slightly-better guess. This is **bootstrapping**.

- **Online, incremental, model-free.** One step is enough — ideal for long or non-terminating tasks.
- **Low variance, some bias.** The target depends on one reward plus a learned value, not a whole noisy trajectory — but it inherits whatever error $V(s_{t+1})$ currently carries.
- In practice TD usually converges *faster* than constant-α MC on stochastic tasks.

Both estimators, the same episodes

RANDOM WALK — PREDICTION FROM THE SAME EPISODES

STEP SIZE **0.10**



Five states, a random walk from **C**, reward 1 only on exiting right — so $V^\pi(s) = i/6$. MC and TD see the *identical* stream and differ only in the target. Watch TD's low-variance target settle while MC's realized returns keep jittering; then raise α and watch MC degrade faster.

The triangle — MC, DP, and TD

Three ways to back up a value; TD is the one that takes from both parents.

	SAMPLES (NO MODEL)	BOOTSTRAPS (USES OWN ESTIMATE)
Dynamic Programming *(Lec 7)*	no — needs full P	yes
Monte Carlo	yes	no — waits for the true return
Temporal Difference	**yes**	**yes**

TD = the model-free-ness of MC + the bootstrapping of DP.

That single inheritance is why TD, not MC, becomes the engine of control. We now turn the prediction crank into a *decision-making* one.

ACT 2

Improvement without a model

Q2. We can score a policy. How do we make it better, still without P?

Why V is not enough — the hidden P

Lecture 7's improvement step, written out:

$$\pi'(s) = \arg\max_a \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma V(s')\big]$$

To turn a *value* into an *action*, you must look one step ahead — and looking ahead needs the model.

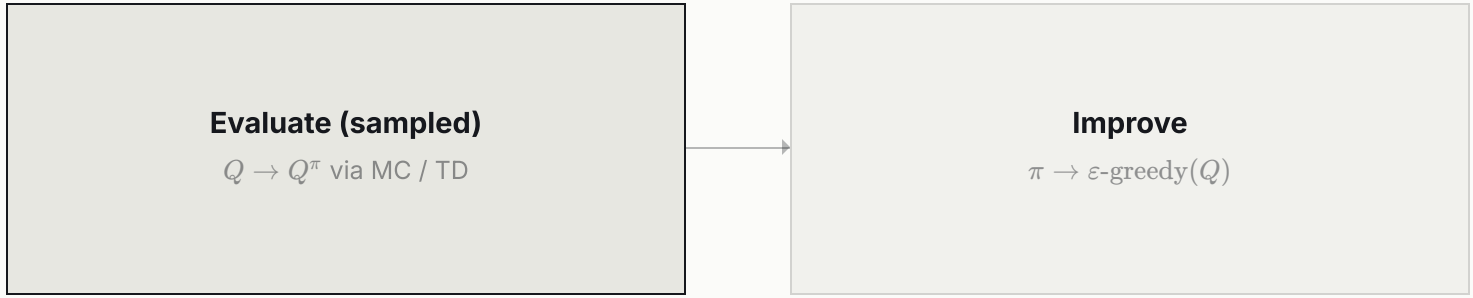
The fix is almost embarrassingly simple. Store the action-value Q instead of V:

$$\pi'(s) = \arg\max_a \; Q(s, a)$$

No sum, no P. **Q already has the look-ahead baked in.** This is *the* reason value-based RL learns Q, not V: Q is the model-free currency of decisions.

Generalized Policy Iteration, now from samples

The Lecture 7 dance is unchanged — evaluate, improve, repeat — but each half is now sampled.



One subtlety the model-based version never faced: to estimate $Q^\pi(s, a)$ from data, *every* (s, a) must actually be tried. A purely greedy policy never tries the actions it currently dislikes — and so never learns they were good.

Improvement now requires exploration. That is the new tax.

The tax, paid — ε-greedy, and the bandit underneath

The simplest way to keep trying everything:

$$a_t = \begin{cases} \arg\max_a Q(s_t, a) & \text{with prob. } 1-\varepsilon \quad \text{(exploit)} \\ \text{a random action} & \text{with prob. } \varepsilon \quad \text{(explore)} \end{cases}$$

This is the **exploration–exploitation trade-off** of the n-armed bandit, now living inside every state. The bandit was RL with the dynamics deleted; here the dynamics are back, but the same dilemma — *spend on what you know, or pay to learn more* — governs every step.

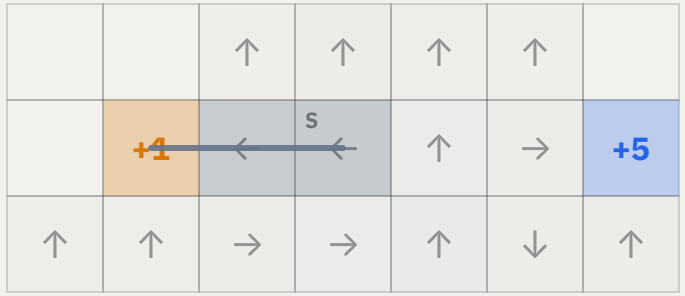
- Decay $\varepsilon \to 0$ (e.g. $\varepsilon = 1/t$) and, with every (s, a) visited infinitely often, the greedy policy in the limit is optimal.
- Hold ε fixed and you keep a permanently-cautious, permanently-curious agent — sometimes exactly what a real system needs.

The tax, seen — what a greedy agent never learns

GENERALIZED POLICY ITERATION — WITH AND WITHOUT EXPLORATION

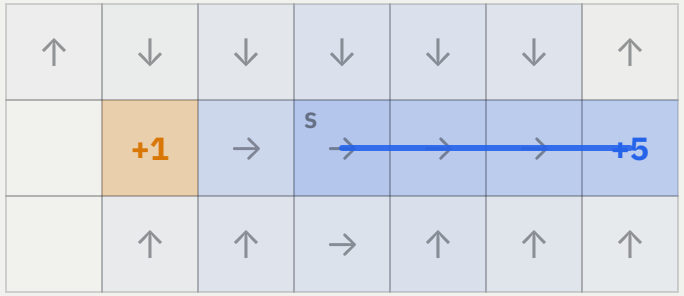
RIGHT E **0.30**

E = 0 — PURE GREEDY



greedy policy **settles for the +1** · tried **57%** of all (s, a)

E-GREEDY — PAYS THE TAX



greedy policy **reaches the +5** · tried **78%** of all (s, a)

near reward **+1** · far reward **+5** · step cost −0.05 · shading = visit count · 400 episodes

Two Q-learning agents, identical but for ε. The greedy one locks onto the near $+1$ it stumbled into first and stops looking — read its coverage figure: the actions it dislikes are never tried, so their values are never corrected. The exploring one pays a little return per episode and finds the far $+5$.

ACT 3

Whose value are we learning?

Q3. The data comes from an exploring policy. Which policy's value does the update actually estimate?

Two control rules, one tiny difference

Q1 Evaluate? → Q2 Improve? → **Q3 Explore?** → Q4 Scale?

Put TD evaluation of Q together with ε-greedy improvement. Two natural updates appear, differing in *one term*:

SARSA — ON-POLICY

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]$$

a' is the action **actually taken** next.

Q-LEARNING — OFF-POLICY

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

$\max$ over **all** next actions, taken or not.

S, **A**, **R**, **S′**, **A′** — SARSA literally uses the next action it sampled. Q-learning replaces that sampled a' with the **best possible** a'. That single $\max$ is the entire on/off-policy distinction.

The 2×2 that organizes everything

Cross the two forks of the whole lecture — *bootstrap?* and *whose policy?*

	NON-BOOTSTRAP (MC)	BOOTSTRAP (TD)
On-policy	MC control	SARSA
Off-policy	off-policy MC	Q-learning

On-policy: the policy that *generates* the data is the policy we *evaluate*. SARSA learns the value of the very (exploring, ε-greedy) behavior it is running.
Off-policy: the behavior policy and the target policy differ. Q-learning *behaves* ε-greedily but *learns* the value of the pure-greedy optimal policy — it studies one policy while living as another.

Why Q-learning is "off-policy," precisely

Look at what each update assumes about the next step.

- SARSA's target uses $Q(s', a')$ where $a' \sim \pi_{\text{behavior}}$ — the **real** next action, exploration and all. It is honest about the policy it runs.
- Q-learning's target uses $\max_{a'} Q(s', a')$ — it *pretends* the next action will be greedy, regardless of the a' actually taken. The behavioral a' is collected, then **ignored**.

THE CLEAN EXTREME — act with $\varepsilon = 1$, i.e. fully random

SARSA learns the value of *acting randomly*. Q-learning learns the value of the *optimal* policy — **while acting randomly**.

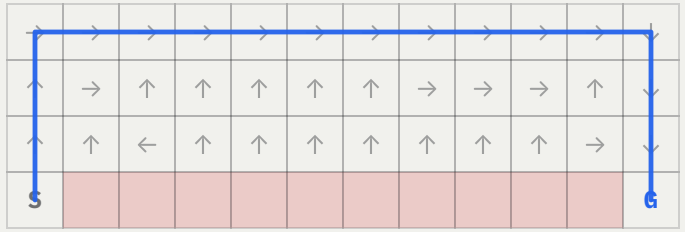
Because the learned target is independent of the behavior, Q converges to Q^* no matter how (sufficiently exploratory) the data was gathered.

The cliff — where the difference becomes visible

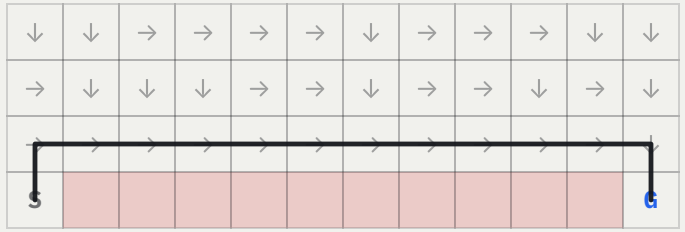
THE CLIFF — ON-POLICY VS. OFF-POLICY CONTROL

E 0.10

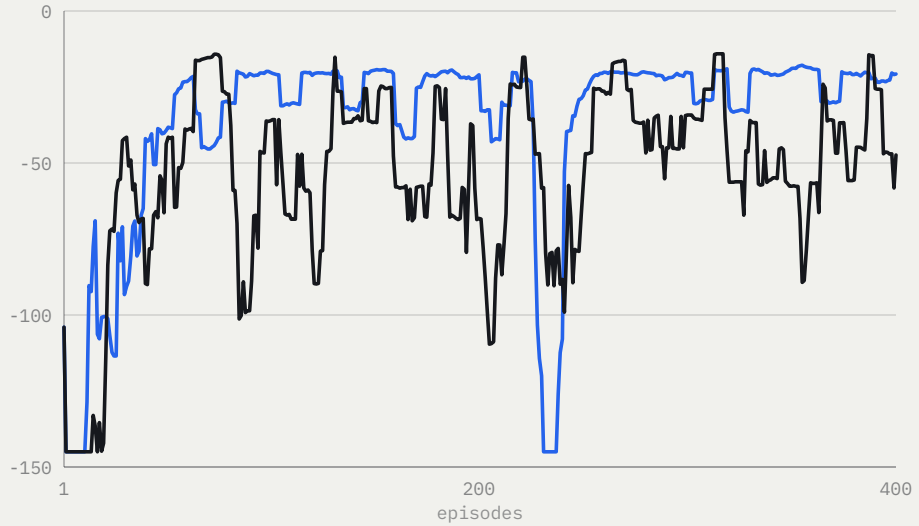
SARSA — GREEDY PATH AFTER 400 EPISODES



Q-LEARNING — GREEDY PATH



ONLINE RETURN PER EPISODE (SMOOTHED)



SARSA (on-policy) Q-learning (off-policy) the cliff — −100 and back to start

Q-learning learns the *optimal* path — along the edge — but it *acts* ε-greedily, and the occasional random step plunges it off the cliff: higher reward in theory, worse online. **SARSA** accounts for its own exploration and learns a **cautious** detour. Neither is "better" — the cliff teaches the design choice.

ACT 4

Scaling past the table

Q4. Everything so far stores one number per (s, a). What happens when the state is an image?

The wall — a table cannot hold the world

Q1 Evaluate? → Q2 Improve? → Q3 Explore? → **Q4 Scale?**

Everything so far stores one number per (s, a). Then comes Atari:

$$\text{state} = \text{four } 84 \times 84 \text{ grayscale frames} \;\Rightarrow\; |\mathcal{S}| \approx 256^{84\times84\times4}$$

more configurations than atoms in the universe. A Q-table is impossible three times over: too many states to *visit*, too many to *store*, and — fatally — no **generalization**: a table says nothing about a state it has never seen.

The machine-learning answer: stop storing, start *approximating*.

$$Q(s,a) \;\approx\; \hat{Q}(s,a;w), \qquad \text{e.g. } \hat{Q}(s,a;w) = w^\top \phi(s,a) \text{ or a neural net}$$

Now "learning" means fitting w — and similar states share answers.

Q-learning as regression — the semi-gradient step

Treat the Bellman target as a label and minimize squared error on each transition:

$$\mathcal{L}(w) = \tfrac{1}{2}\Big(\underbrace{r + \gamma \max_{a'} \hat{Q}(s', a'; w)}_{\text{target}} - \hat{Q}(s, a; w)\Big)^2 \;\Rightarrow\; w \leftarrow w + \alpha\, \delta\, \nabla_w \hat{Q}(s, a; w)$$

with TD error $\delta = r + \gamma \max_{a'} \hat{Q}(s', a'; w) - \hat{Q}(s, a; w)$.

It is called a **semi-gradient** method for an honest reason: we differentiate the prediction $\hat{Q}(s, a; w)$ but *treat the target as fixed*, even though it too depends on w. We are chasing a target that moves when we move.

In the tabular world that chase still converged. With function approximation, it can **diverge**. Tabular Q-learning's guarantees do not survive the jump — and we must understand why before trusting a network.

The deadly triad — why naive deep Q-learning blows up

Three ingredients are each harmless alone. Together they can make value estimates diverge.

Function approximation
errors spread across states

→

Bootstrapping
target built from its own moving estimate

→

Off-policy
data from a policy $\neq$ the one evaluated

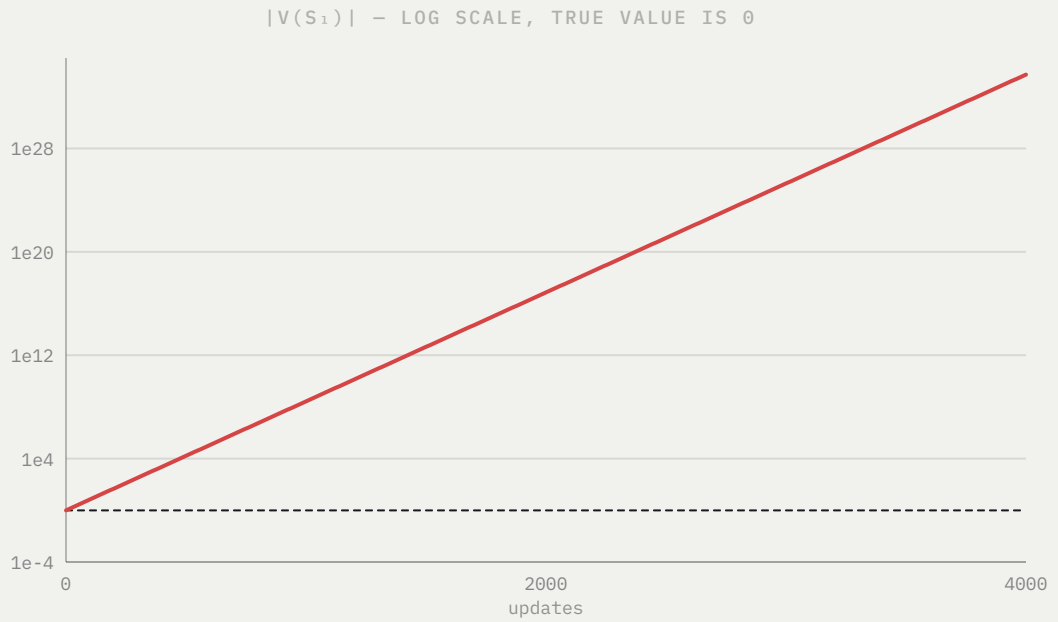
Q-learning with a neural net has *all three*. The data are also strongly **correlated** (consecutive frames) and the target **shifts** on every update. Plain SGD on $\mathcal{L}(w)$ fights itself.

The breakthrough was not a new objective — it was **two engineering tricks** that tame the triad.

The triad, switched on and off

TWO STATES, ZERO REWARDS, TRUE VALUE = 0 — WHAT COULD GO WRONG

DIVERGING
$|V(s_1)|$ = 5.2e+33



Two states, every reward 0, so the true value is 0 everywhere. With all three switches on, $|V(s_1)|$ grows without bound. Turn **any one** of them off — use a table, use the return instead of a bootstrap, or update on-policy — and the same algorithm collapses to 0. *(Tsitsiklis & Van Roy, 1997)*

DQN — the two tricks

Mnih et al., 2013; Nature 2015

Keep Q-learning's update. Add two stabilizers.

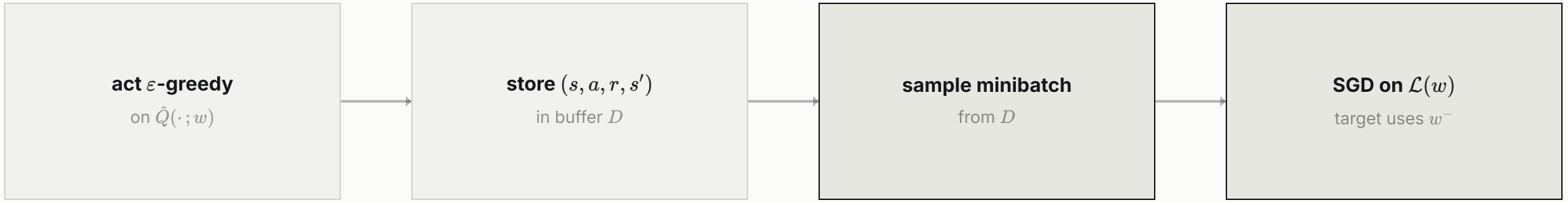
- **Experience replay.** Store transitions (s, a, r, s') in a buffer D; train on *random minibatches* from it. This **breaks correlation** between consecutive samples and reuses each experience many times — restoring something close to i.i.d. data.
- **Target network.** Compute the target with a *frozen* copy w^-, synced to w only every C steps:

$$\mathcal{L}(w) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(r + \gamma \max_{a'} \hat{Q}(s', a'; w^-) - \hat{Q}(s, a; w)\right)^2\right]$$

Now the target stops moving while we chase it — the **moving-target** pathology is suppressed.

Result: one architecture, one set of hyperparameters, **human-level play across dozens of Atari games** from raw pixels. The Bellman equation, untouched; the model, never learned; the table, replaced by a convnet.

The DQN loop — everything from today, assembled



every C steps: $w^- \leftarrow w$

Trace the lineage of each box: *act ε-greedy* (Act 2's exploration tax), *the $\max$ in the target* (Act 3's off-policy Q-learning), *bootstrap target* (Act 1's TD), *buffer + frozen w^-* (Act 4's triad fixes). One loop holds the whole lecture.

What comes after DQN — one slide of horizon

The two tricks opened a decade of refinements, each patching a named flaw:

- **Double DQN** — the $\max$ over-estimates; decouple action-selection from evaluation. *(van Hasselt et al., 2016)*
- **Dueling networks** — split $\hat{Q}$ into state-value + advantage; learn "which states matter" separately. *(Wang et al., 2016)*
- **Prioritized replay** — sample high-error transitions more often. *(Schaul et al., 2016)*
- **Rainbow** — combine them; the sum beats every part. *(Hessel et al., 2018)*

Every one keeps the same skeleton: *sampled Bellman backup* + Q *for model-free greed* + *a stabilizer*. None brings back the model — and none works in **continuous action spaces**, where $\max_{a'}$ itself becomes intractable.

That wall — the $\max$ over a continuum — is exactly where Lecture 9 begins.

CLOSING

Closing

Where we are — the cell, filled

	MODEL-BASED	MODEL-FREE
Static, single	optimization *(Lec 1)*	bandit *(Lec 8, intro)*
Dynamic, discrete, single	MDP / DP *(Lec 7 ✓)*	value-based RL ***(Lec 8 ✓)***
Dynamic, continuous, single	optimal control *(Lec 9)*	policy-based RL *(Lec 10)*

One repeated move, three times over: **model → sample**, **wait → bootstrap**, **table → function**.

Lecture 7 could solve the Bellman equation, because it owned the world. Lecture 8 *learned* to solve it.

by sampling the expectation, storing Q instead of V, and replacing the table with a network it could trust.

Questions?

From a windy gridworld to human-level Atari with one update rule — the distance between *knowing* the dynamics and *never needing to*. What stays fixed through all of it is the equation Bellman wrote; we only ever changed what we were allowed to look up.

APPENDIX

Backup slides

Complete arguments, kept out of the narrative.

Backup 1 — MC vs. TD, the bias–variance ledger

Both estimate $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ from samples; they differ in *what* they sample.

MONTE CARLO TARGET $G_T = \sum_K \Gamma^K R_{T+K+1}$

- *Bias:* none — $\mathbb{E}[G_t] = V^\pi(s_t)$ exactly.
- *Variance:* high — a function of **many** random rewards and transitions along the whole episode.

TD(0) TARGET $R_{T+1} + \Gamma V(S_{T+1})$

- *Bias:* present — uses the current (imperfect) estimate $V(s_{t+1})$; unbiased only at the true V^π.
- *Variance:* low — depends on **one** reward and one transition.

THE BRIDGE — N-STEP AND TD(Λ)

Interpolate by bootstrapping after n steps, $G_t^{(n)} = r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$; $n = 1$ is TD(0), $n = \infty$ is MC. TD(λ) averages all n geometrically. The whole family is one dial between *low-bias / high-variance* and *higher-bias / low-variance*.

Backup 2 — when does tabular Q-learning converge?

Claim *(Watkins & Dayan, 1992)*: tabular Q-learning converges to Q^* with probability 1, provided

- every state–action pair (s, a) is visited **infinitely often** (the role of exploration, e.g. $\varepsilon > 0$), and
- the step sizes satisfy the Robbins–Monro conditions $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$.

Why it holds (sketch). The Bellman optimality operator $(\mathcal{T}Q)(s,a) = \mathbb{E}_{s'}[\, r + \gamma \max_{a'} Q(s', a') \,]$ is a γ-contraction in the sup-norm, so it has a unique fixed point Q^*. The Q-learning update is a **stochastic approximation** of $\mathcal{T}$: each step replaces the expectation by one sample. Robbins–Monro step sizes average out the sampling noise; infinite visitation guarantees every entry keeps being corrected. Together they drive $Q \to Q^*$.

What breaks under approximation. Replace the table by $\hat{Q}(\cdot; w)$ and the update is no longer a contraction in any norm the parameters respect — the projection onto the function class can *expand* distances. That is the formal root of the deadly triad, and the reason DQN needs its two crutches.

Backup 3 — the function-approximation gradient, in full

Linear case $\hat{Q}(s,a;w) = w^\top \phi(s,a) = \sum_k w_k \phi_k(s,a)$. Per-transition squared TD error:

$$\mathrm{Err}(w) = \tfrac{1}{2}\Big(r + \gamma \max_{a'} \hat{Q}(s',a';w) - \hat{Q}(s,a;w)\Big)^2$$

Differentiate, holding the target fixed (semi-gradient):

$$\frac{\partial\, \mathrm{Err}}{\partial w_k} = -\Big(r + \gamma \max_{a'} \hat{Q}(s',a';w) - \textstyle\sum_j w_j \phi_j(s,a)\Big)\phi_k(s,a)$$

$$\Rightarrow \quad w_k \leftarrow w_k + \alpha \Big(r + \gamma \max_{a'} \hat{Q}(s',a';w) - \hat{Q}(s,a;w)\Big)\phi_k(s,a)$$

In vector form $w \leftarrow w + \alpha\, \delta\, \phi(s,a)$ — the tabular update is the special case $\phi(s,a) = \mathbf{e}_{(s,a)}$ (a one-hot indicator), which is why tabular RL is "just" function approximation with the most local possible features. For a neural net, ϕ becomes $\nabla_w \hat{Q}(s,a;w)$ and the same line holds.

Backup 4 — model-based RL, the alternative we set aside

The branch we acknowledged in Act 0 and walked away from. **Estimate the model** from the same stream of experience:

$$\hat{P}(s' \mid s, a) = \frac{\#(s, a, s')}{\#(s, a)}, \qquad \hat{R}(s, a, s') = \text{mean of } r \text{ over } (s, a, \cdot, s')$$

Then solve the estimated MDP with any Lecture 7 method (value / policy iteration).

- *Pro — sample efficiency:* a learned model can be "replayed" indefinitely; each real transition informs the value of *many* states through planning.
- *Con — model bias:* errors in $\hat{P}, \hat{R}$ compound through planning; you optimize for a world that is slightly wrong.
- *Con — cost:* estimating a full kernel is far more than estimating the Q^* we actually act on.

This is why model-free dominated the deep-RL era — and why **Lecture 11** returns to model-based methods only once it can make the learned model carry its weight (planning, Dyna, differentiable control).